

EDUGENIE

Phase 7 – Project Documentation

AI-Powered Learning Assistant

Project Documentation Report

Prepared for Academic Project Submission

1. Introduction

EduGenie is an AI-powered learning assistant designed to support students during their learning process. The system uses an AI service to provide educational assistance through features such as question answering, topic explanation, summarization, quiz generation, and learning-path generation.

The project is intended to provide a simple interface through which students can enter learning-related requests and receive useful, structured responses. This document describes the project, its objectives, major modules, technology stack, working process, testing approach, and expected benefits.

2. Problem Statement

Students often need quick explanations, summaries, practice questions, and guidance while studying. Traditional learning resources may require students to search through multiple sources before finding an explanation suitable for their level. EduGenie aims to provide these learning-support functions through one AI-powered application.

3. Project Objectives

- Provide an AI-based assistant for educational questions.
- Generate simple explanations for difficult topics.
- Create concise summaries of learning content.
- Generate quizzes for practice and self-assessment.
- Create structured learning paths for selected goals.
- Provide a simple and user-friendly interface for students.
- Demonstrate the integration of a web application with an AI service.

4. Scope of the Project

The scope of EduGenie covers the development of a web-based learning assistant with multiple AI-supported learning modules. The project focuses on educational assistance and does not replace teachers, institutional learning systems, or professional academic assessment.

- Student question answering
- Topic explanation
- Content summarization
- Quiz generation
- Learning-path generation
- Basic input validation and error handling
- Frontend and backend integration

5. System Overview

EduGenie follows a client-server architecture. The user interacts with the web interface, the frontend sends the request to the backend, and the backend communicates with the configured Gemini AI service. The generated response is returned to the frontend and displayed to the user.

The main processing flow is:

1. Student opens the EduGenie application.
2. Student selects a learning feature.
3. Student enters a question, topic, or learning goal.

- 4. The frontend sends the request to the backend.
- 5. The backend processes the request and sends it to the AI service.
- 6. The AI service generates the requested educational response.
- 7. The backend returns the response to the frontend.
- 8. The result is displayed to the student.

6. Main Modules

Module	Purpose	Example Use
Q&A	Answer educational questions.	Ask: What is artificial intelligence?
Explanation	Explain a topic in understandable language.	Explain machine learning simply.
Summary	Generate a concise summary.	Summarize study content.
Quiz	Generate practice questions.	Create a quiz on computer science.
Learning Path	Provide ordered learning steps.	Create a path to learn Python.

7. Technology Stack

Technology	Purpose
Python	Application development
FastAPI	Backend API and web application framework
Gemini API	AI-powered response generation
HTML	Web page structure
CSS	Web page styling
JavaScript	Frontend interaction and API communication
pytest	Testing
Git / GitHub	Version control and project submission

8. Project Structure

The development phase is organized so that the application code, configuration, modules, templates, static files, and tests are separated into appropriate folders.

```
edugenie/  
├── app/  
│   ├── __init__.py  
│   ├── main.py  
│   ├── config.py  
│   ├── gemini_client.py  
│   ├── modules/  
│   │   ├── qna.py  
│   │   ├── explanation.py  
│   │   ├── summary.py  
│   │   ├── quiz.py  
│   │   └── learning_path.py  
│   ├── templates/  
│   │   └── index.html  
│   ├── static/  
│   │   ├── style.css  
│   │   └── app.js  
└── tests/  
    └── test_api.py
```

9. Functional Requirements

- The system shall allow the user to access the EduGenie interface.
- The system shall accept educational questions and requests.
- The system shall generate AI-based answers for valid questions.
- The system shall provide topic explanations.
- The system shall generate summaries.
- The system shall generate quizzes.
- The system shall generate learning paths.
- The system shall handle invalid or empty input appropriately.
- The system shall display generated results to the user.

10. Non-Functional Requirements

- Usability: The interface should be simple enough for students to understand.
- Performance: Normal requests should receive a response without unnecessary delay.
- Reliability: The application should handle errors without crashing.
- Maintainability: Modules should be separated so that features can be updated independently.
- Security: API credentials should be stored in environment configuration and not exposed in the public repository.

11. Working Process

EduGenie works by accepting a user request through the web interface and forwarding it to the backend. The backend selects the appropriate module and prepares the request for the configured AI service. The generated response is then returned to the frontend for display.

- Input collection
- Request validation
- Backend processing
- AI service request
- Response generation
- Result display

12. Testing and Validation

The project testing phase includes test cases for application launch, Q&A, explanation, summary, quiz generation, learning paths, validation, API response, error handling, navigation, and frontend-backend integration.

The Test_Cases.xlsx file contains the documented test cases and recorded results for the project testing phase. Testing is intended to verify that the major application functions work as expected.

13. Benefits of EduGenie

- Provides quick access to AI-supported learning assistance.
- Combines several learning-support functions in one application.
- Can help students understand topics in simpler language.
- Supports self-practice through quiz generation.
- Provides structured learning guidance through learning paths.
- Demonstrates practical use of AI in an educational application.

14. Limitations

- The quality of responses depends on the AI service and the user's input.
- The application requires a configured AI API connection for AI-generated responses.
- AI-generated information should be reviewed by students and used along with reliable learning resources.
- The project is an academic prototype and may require additional development for large-scale production use.

15. Future Enhancements

- User accounts and personalized learning profiles.
- Progress tracking and learning history.
- More subject-specific learning modules.
- Automatic quiz scoring and performance analytics.
- Multilingual learning support.
- Voice-based question and answer functionality.
- Teacher or administrator dashboard.
- Improved accessibility and mobile-responsive design.

16. Project Deliverables

- Brainstorming and Ideation document
- Requirement Analysis document
- Project Design document and diagrams
- Project Planning document
- Project Development source code
- Test Cases Excel file and Testing Report
- Project Documentation report
- Project Demonstration video link

17. Conclusion

EduGenie demonstrates how an AI-powered application can be used to support common student learning activities. By combining question answering, explanations, summaries, quizzes, and learning paths in a single system, the project provides a practical example of AI integration in education.

The project documentation, development files, testing records, and demonstration materials together provide a structured record of the development process from planning through final demonstration.